UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Denkleiers • Leading Minds • Dikgopolo tša Dihlalefi

COS 221 Practical Assignment 4

- Date Issued: **15 April 2026**
- Date Due: **29 April 2026** before **11:00 AM**
- Submission Procedure: **Upload to ClickUP**
- This assignment consists of **6 tasks** for a total of **90 marks** plus **15 bonus marks**.

1 Introduction

You are required to use the Chinook Sample Music Store database. Chinook provides a well-defined, fully normalised database modelling a digital music store. It covers concepts such as artists, albums, tracks, invoices, customers, and employees, all of which you would expect in a real-world e-commerce music platform.

In the database you will find tables such as `Artist`, `Album`, `Track`, `Genre`, `MediaType`, `Playlist`, `Customer`, `Employee`, `Invoice`, and `InvoiceLine`. Explore the tables to understand what the database represents before beginning the tasks.

After successful completion of this assignment you should be able to:

- Model an existing normalised database;
- Analyse and understand database extensions from external sources;
- Build a Graphical User Interface (GUI); and
- Utilise the GUI to query and manipulate a relational database.

2 Constraints

1. You must complete this assignment individually or in pairs.
2. You may ask the Teaching Assistants for help, but they will not provide solutions.
3. The PDF, database dump, and GUI will be marked.
4. The GUI interfaces:
   (a) If the GUI runs and performs as expected, it will receive full marks.
   (b) If it runs but does not perform as required, it will receive partial marks.
   (c) If it does not run, marks will be awarded based on intended functionality.
5. You need to use MariaDB or MySQL Workbench.
6. You may use any text editor or IDE on any operating system (e.g., pgAdmin).
7. Hint: You are **NOT REQUIRED** to use Git for this practical. However, Git will be required for Practical Assignment 5, so it is recommended that you start learning Git and GitHub.

3 Submission Instructions

You are required to upload a single archive that includes the following files:

- An archive containing your GUI application project.
- A PDF containing the answers to the tasks where required.
- A `readme.txt` file explaining how to build the project, connect the application to the database, and run the application.

Upload your archive to ClickUP. No late submissions will be accepted, so make sure you upload in good time.

4 Online Resources

The following resources will help with the practical and creating your GitHub:

Git: `https://git-scm.com`

GitHub: `https://github.com`

MariaDB: `https://mariadb.com`

MariaDB Getting Started: `https://mariadb.com/get-started-with-mariadb/`

5 Rubric for Marking

Task 1: Obtain the Database	**4**
Database downloaded and imported successfully	1
Database named correctly with student number(s)	1
Database is fully functional and all tables are present	2
Task 2: Pre-Packaged Application (DB Tool)	**4**
Name and specification of chosen DB Tool	2
Screenshot showing imported tables with database name visible	2
Task 3: (E)ER-Diagram	**6**
Generated (E)ER-diagram included in PDF submission	2
Invoice table: data held and data types correctly explained	2
Invoice table: constraints and relationships correctly explained	2
Task 4: Graphical User Interface	**71**
4.1 Employees Tab Listing	*6*
Table component present and displays employee data	2
Correct columns shown (name, title, city, country, phone, supervisor)	2
Data retrieved correctly across Employee table	2
4.2 Employees Tab Filter	*4*
Text box present on the Employees tab	1
Filtering by employee name works correctly	2
Filtering by city works correctly	1
4.3 Tracks Tab – Add New Track	*8*
Button present that opens the add-track popup dialog	1
Dropdown for Album populated from the database	2
Dropdowns for Genre and MediaType populated from the database	2
New track saved correctly to the database	2
Table reloads and displays the new entry after saving	1
4.4 Report Tab – Genre Revenue Report	*6*
Report generated each time the tab is opened	2
Genre name and total revenue returned correctly	2
Results ordered from highest to lowest revenue	2
4.5 Notifications Tab – Customer CRUD	*18*
Customer list displayed (Read)	2
Create new customer works and reflects in the table	4
Update existing customer works and reflects in the table	5
Delete customer works and reflects in the table	4
Required fields exposed (name, email, phone, country)	3
4.6 Notifications Tab – Inactive Customers	*5*
Table present listing customers with no invoices or stale invoices	2
Two-year threshold applied correctly in the query	2
Searchable list of all customers (past and present)	1
4.7 Customer Recommendations Tab	*24*
Customer dropdown present and updates all displayed data dynamically	3
Spending summary displayed (total spent, total purchases, last purchase date)	4
Favourite genre determined correctly via SQL aggregation and joins	5
Recommendation engine filters out already-purchased tracks	6
Recommended tracks belong to the customer's preferred genres	4
Results presented clearly and in a user-friendly layout	2
Task 5: Securing Credentials	**5**
Application reads connection details from environment variables	2
All six required environment variables used correctly	3
Total	**90**

Bonus Task 6: Git & GitHub	**7**
Individual: 8 or more commits from the student	3
Pair: 8 or more commits from each member	4
Bonus Task 7: Advanced SQL	**3**
Advanced or non-standard SQL documented and correctly used in the application	3
Bonus Task 8: Overall Presentation	**5**
GUI is visually polished, intuitive, and demonstrates good design practice	5
Total (including bonus)	**105**

6 Assignment Instructions

Task 1: Obtain the Database ..... (4 marks)

Download the Chinook database from the official GitHub repository: `https://github.com/lerocha/chinook-database/releases`

Download the `Chinook_MySql.sql` script. Import the database into your MariaDB instance and name it `uXXXXXXXX_chinook`, where `XXXXXXXX` is your student number.

If working in pairs, name it `uXXXXXXXX_uYYYYYYYY_chinook`. Verify that the import is correct before proceeding.

Task 2: Choose a Pre-Packaged Application (DB Tool) ..... (4 marks)

2.1 Specify the pre-packaged application you have chosen. (2)

2.2 Provide a screenshot of the application showing the imported database tables. Your database name with your student number must be clearly visible in the screenshot. (2)

Task 3: (E)ER-Diagram ..... (6 marks)

3.1 Generate an (E)ER-diagram using your chosen application. Include the diagram in your PDF submission. (2)

3.2 Explore the `Invoice` table structure. Explain the data this relation holds, the data types, constraints present, and how it links to other relations in the schema. (4)

Task 4: Graphical User Interface ..... (71 marks)

4.1 **Employees Tab Listing** Add a table component showing at a minimum: first name, last name, title, city, country, phone, reports-to (supervisor name), and whether they are active. (6)

4.2 **Employees Tab Filter** Add a text box allowing the user to filter results by employee name or city in real time (or on button click). (4)

4.3 **Tracks Tab: Add New Track** Add a button that opens a popup dialog to add a new track. Include dropdown menus populated from the database for Album, Genre, and MediaType. After saving, reload the table to display the new entry. (8)

4.4 **Report Tab: Genre Revenue Report** Each time the Report tab is opened, generate a report showing total revenue per genre. Return genre name and sum of invoice totals, ordered from highest to lowest. (6)

4.5 **Notifications Tab: Customer CRUD** Implement Create, Read, Update, and Delete functionality for customers. At minimum expose: first name, last name, email, phone, and country. (18)

4.6 **Notifications Tab: Inactive Customers** Add a searchable table listing customers who have no invoices or whose most recent invoice is older than 2 years. (5)

4.7 **Customer Recommendations Tab** (24)

Design and implement a "Customer Insights & Recommendations" tab for the music store application. This tab should provide advanced analytics and personalised insights based on customer purchase history.

- **Customer Selection:** Provide a dropdown menu that allows the user to select a specific customer from the database. All displayed insights should update dynamically based on the selected customer.
- **Spending Summary:** Display a summary of the customer's purchasing behaviour, including:
  - Total amount spent across all purchases
  - Total number of purchases made
  - Date of the most recent purchase
- **Favourite Genre Analysis:** Using SQL aggregation and joins, determine the customer's most frequently purchased music genre. This should be derived from historical purchase data by grouping purchases according to genre and identifying the highest frequency.

- **Track Recommendation Engine:** Implement a basic recommendation system that suggests new tracks for customers, which employees can recommend. Recommendations should be generated based on:
  - Genres most frequently purchased by the customer
  - Similar tracks within those preferred genres that the customer has not yet purchased

Hint: Use a build tool such as Maven, Ant, or Gradle, or manually add the MariaDB/MySQL Connector/J JAR to your classpath.

Database Connectivity using JDBC Driver

In this practical, the Java application requires communication with a relational database such as MySQL or MariaDB. However, Java does not provide built-in support for connecting directly to these databases.

To enable this communication, a **JDBC (Java Database Connectivity) driver** must be included, specifically the **MySQL/MariaDB Connector/J**. This driver acts as a bridge between the Java application and the database, allowing SQL operations to be executed.

The JDBC driver enables the application to:

- Send SQL queries to the database
- Retrieve results from database tables
- Perform insert, update, and delete operations

Without this driver, the Java application cannot establish a connection to the database or execute SQL statements.

Obtaining the Driver

The official JDBC drivers can be downloaded from the following sources:

- MySQL Connector/J: `https://dev.mysql.com/downloads/connector/j/`
- MariaDB Connector/J: `https://mariadb.com/downloads/connectors/java/`

Adding the Driver to the Project

There are two main ways to include the JDBC driver in a Java project:

1. Using a Build Tool (Recommended) When using build tools such as Maven, Gradle, or Ant, the dependency is added to the project configuration file. The build tool automatically downloads and manages the required library.

```
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.0.33</version>
</dependency>
```

2. Manual Installation (Classpath Method) If a build tool is not used, the driver can be added manually by:
   - Downloading the `.jar` file
   - Adding it to the project's classpath

This allows the Java Virtual Machine to locate and use the driver at runtime.

Task 5: Securing Credentials ..... (5 marks)

Storing database credentials in plain-text configuration files creates security risks. Modify your application to read connection details from the following environment variables:

- CHINOOK_DB_PROTO
- CHINOOK_DB_HOST
- CHINOOK_DB_PORT
- CHINOOK_DB_NAME
- CHINOOK_DB_USERNAME
- CHINOOK_DB_PASSWORD

Task 6: Bonus Marks ..... (15 marks)

6.1 Each member must commit code using Git with more than 8 commits on GitHub. (7)

6.2 Document any advanced or non-standard SQL used in your application in your PDF. SQL from lectures 1–17 is considered standard. (3)

6.3 Ensure your GUI is visually polished, intuitive, and demonstrates good design practice. Marks will be awarded for overall presentation quality, layout consistency, and user experience. (5)